

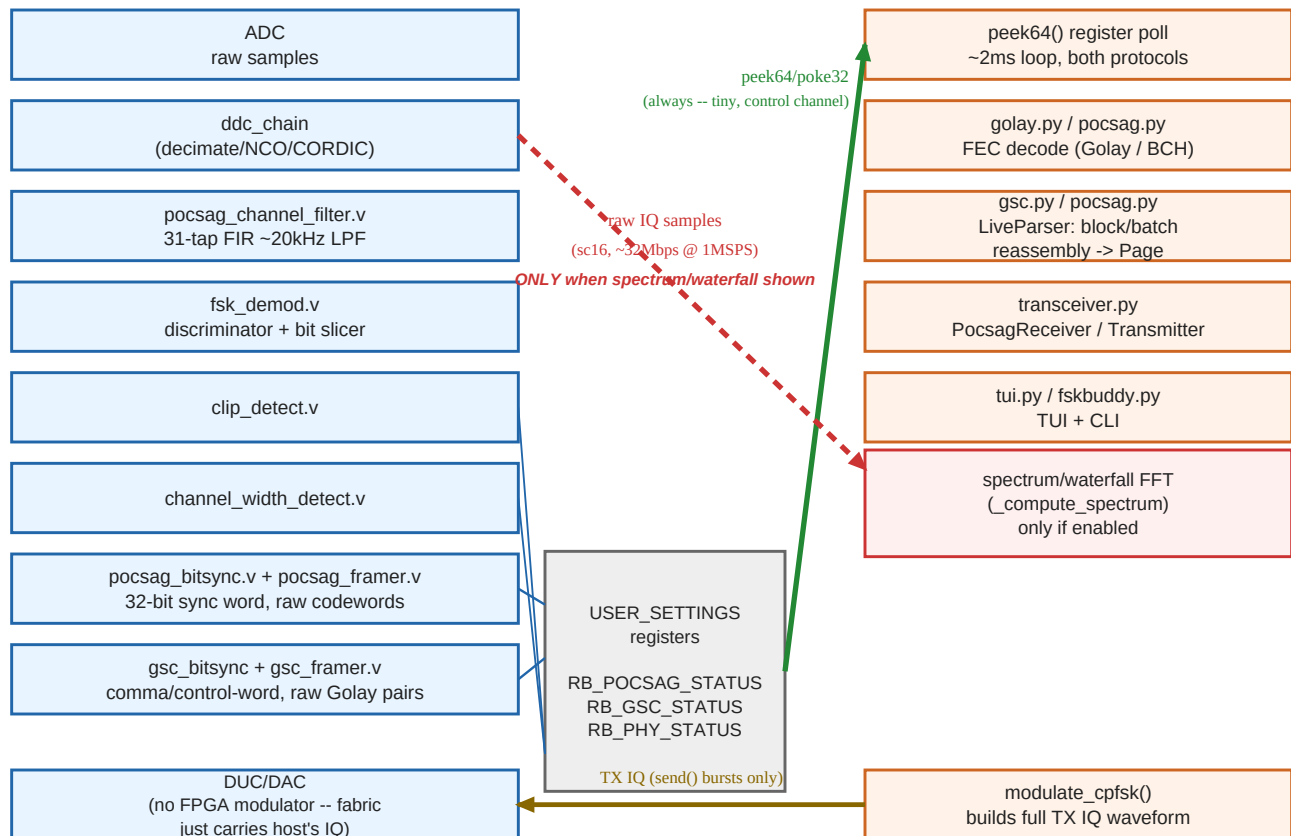
FSK Buddy

What runs on the FPGA, what runs on the host, and what actually crosses USB -- with and without the spectrum/waterfall display

Architecture at a glance

FPGA FABRIC (radio_200)

HOST (Python)



Solid green: small register reads/writes over UHD's control-packet channel -- present in every mode, whether or not the spectrum/waterfall display is on. Dashed red: the continuous raw-IQ sample stream over the USB bulk endpoint -- present only when the spectrum scope or waterfall panel is actually shown. Solid gold: outbound TX IQ, sent only for the duration of an actual transmit burst, not continuous.

- Register reads (peek64/poke32) -- always present, tiny
- Raw IQ samples -- only when spectrum/waterfall is enabled, ~32Mbps
- TX IQ waveform -- bursty, only while actually sending

What runs on the FPGA (fabric)

Everything time-critical and per-sample lives in the custom `radio_200` fabric image, not in Python. This is real-time DSP running at the radio's own sample clock, independent of whatever the host is doing.

ddc_chain — Decimates the raw ADC stream down to the working sample rate (NCO tuning, CORDIC mixer, CIC + halfband decimators) -- stock UHD/B200mini logic, unmodified except for one new gating signal (see "run_rx_fabric" below).

pocsag_channel_filter.v — A 31-tap FIR lowpass (~20kHz cutoff) ahead of the discriminator. Without it, standard POCSAG deviation doesn't decode at all on real RF -- the discriminator sees the DDC's full output bandwidth as noise instead of the pager channel's actual ~15-20kHz width.

fsk_demod.v — A delay-and-multiply FSK discriminator -- the actual bit slicer. Shared by both protocols; also exposes a raw discriminator-swing output (`disc_out`) that `channel_width_detect.v` uses.

clip_detect.v — Watches raw I/Q magnitude for front-end saturation (RX gain set too hot) directly in fabric -- a free-running counter, no per-sample Python work.

channel_width_detect.v — Classifies the RX channel as 12.5kHz ("narrow") or 25kHz ("wide") by estimating FSK deviation from the discriminator's own statistical swing, normalized by signal power -- no second bandpass filter needed.

pocsag_bitsync.v + pocsag_framer.v — POCSAG bit-timing recovery and batch/frame synchronization -- finds the 32-bit sync word (Hamming-tolerant correlation), shifts out raw 32-bit codewords. FEC (BCH) decode is NOT done here -- codewords go out raw.

gsc_bitsync + gsc_framer.v — GSC's equivalent: a second bitsync instance at GSC's own 600-baud rate, plus a framer that locks onto the control word's pattern and tracks comma-delimited blocks. Also ships raw (undecoded) Golay codeword pairs out -- FEC (Golay) decode is host-side.

All of the above are gated by a dedicated `run_rx_fabric` signal (`pocsag_en | gsc_en`), deliberately independent of whether the host is actually streaming/draining raw samples over USB -- the whole chain keeps running and decoding off this gate alone. There is **no FPGA-side transmitter/modulator** -- TX is entirely host-generated IQ (see below) that the DUC/DAC just carries.

What runs on the host (Python)

Everything that isn't strictly real-time-per-sample is in Python -- protocol logic, forward error correction, message assembly, the UI, and (for TX) building the actual RF waveform.

pocsag.py — BCH(31,21) decode with single-bit correction, numeric/alphanumeric message packing, batch/preamble assembly, and LiveParser -- turns a stream of raw codewords (read from the FPGA's registers) into Page objects.

golay.py / gsc.py — Golay(23,12,7) decode, GSC's own framing/message assembly, and its own LiveParser -- the same role as `pocsag.py`, for the GSC codeword pairs the FPGA hands over raw.

modem.py — Shared UHD plumbing -- opening the device, and `modulate_cpfsk()`, which builds the ENTIRE outbound TX IQ waveform in Python (continuous-phase FSK, sample by sample) before handing it to UHD to stream out. There is no FPGA-side modulator to offload this to.

transceiver.py — PocsagReceiver/PocsagTransmitter/PocsagTransceiver -- the reusable API both the TUI and the CLI are built on. Owns the register-poll loop, the optional spectrum FFT, and TX bursts.

tui.py / fskbuddy.py — The interactive TUI and the combined CLI entry point.

What actually crosses USB

This is the part that changed recently, and it's worth being precise about: the FPGA-side work above runs regardless of what the host does with USB -- it always has, since Stage 1. What changed is whether the host bothers to stream raw samples down at all, which is a completely separate question from whether decode works.

Without spectrum or waterfall (register-poll-only)

This is `fskbuddy.py send/listen` always, and `tui --no-spectrum --no-waterfall`. `PocsagReceiver` never issues `stream_cmd(start_cont)` and never calls `rx_streamer.recv()` -- the only USB traffic is `peek64/poke32` register reads/writes, polled roughly every 2ms.

```
peek64() reads RB_POCSAG_STATUS or RB_GSC_STATUS (whichever protocol is active) plus
RB_PHY_STATUS every ~2ms loop iteration. Each read/write is a small UHD control-packet
transaction -- not the bulk sample-streaming endpoint an RX packet uses.
```

Confirmed by construction: `rx_streamer.recv()` is provably never called in this mode (`self._streaming == False` the whole session).

"How many bytes/sec that leaves" is a bound, not a measured capture (this environment has no `usbmon` access) -- at up to ~500-1000 tiny register transactions/sec, a generous estimate stays well under 2Mbps.

With spectrum or waterfall shown

As soon as either panel is enabled, `PocsagReceiver` issues `stream_cmd(start_cont)` and continuously calls `rx_streamer.recv()` to drain the full-rate raw IQ stream -- because the FFT that draws those panels needs real sample content, and there's no way to get that except by actually streaming it.

```
At 1MSPS with the sc16 wire format (2 bytes I + 2 bytes Q = 4 bytes/sample):
1,000,000 samples/sec x 4 bytes = 4MB/s = 32Mbps, continuous
```

This is IN ADDITION to the register reads above, not instead of them -- decode still happens entirely via FPGA registers either way. The raw IQ samples are used ONLY for the spectrum bars and waterfall's FFT input; they play no role in actually decoding a page.

Side by side

Mode	USB traffic	What crosses
send / listen (always)	< ~2Mbps (bound, not measured)	peek64/poke32 register reads only
tui, both panels hidden	< ~2Mbps (bound, not measured)	peek64/poke32 register reads only
tui, spectrum or waterfall shown	~32Mbps continuous	raw IQ (sc16) + register reads
any mode, while actually sending	+ ~32Mbps for the burst duration	outbound TX IQ (bursty, not continuous)

Root cause of why this split is even possible: the FPGA's decimator (`ddc_chain`) used to share its own `run` enable with the same signal (`run_rx`) that gates the host-facing USB packet path -- so decode silently stalled whenever the host stopped draining USB, even though none of the custom PHY logic actually needed the USB link. A dedicated `run_rx_fabric` signal in `radio_legacy.v` now drives only the decimator, completely independent of the original `run_rx` (which still gates the USB packet path exactly as before) -- confirmed on real hardware: with zero `recv( )` calls after an initial `stream_cmd`, pages still decode purely off register polling.